



**Politecnico
di Torino**

POLITECNICO DI TORINO

Corso di Laurea in Matematica per L'Ingegneria

Tesi di Laurea

Metodi di Machine Learning basati su Random Radial Basis Function

Relatore:

Prof. Tommaso Vanzan

Candidato:

Francesco Foglia

Anno Accademico 2025/2026

Sommario

Questo elaborato di tesi esplora l'applicazione delle reti a funzione di base radiale (Radial Basis Function o RBF) nell'ambito dell'apprendimento supervisionato, con particolare attenzione al problema della regressione non parametrica. A differenza degli approcci tradizionali che si affidano a complessi algoritmi di ottimizzazione iterativa (come la discesa del gradiente), il presente lavoro analizza le reti RBF interpretandole in modo rigoroso come modelli statistici lineari nei parametri. Questa prospettiva permette di derivare la stima ottimale dei pesi sfruttando l'efficienza e la stabilità degli strumenti dell'algebra lineare, garantendo la convergenza verso un minimo globale dell'errore.

Per contrastare il fenomeno dell'overfitting e gestire l'incertezza ai dati di addestramento, lo studio approfondisce l'utilizzo della regolarizzazione (Ridge Regression), analizzandone sia la formulazione globale che quella locale. Un nucleo centrale della trattazione è dedicato al problema della *model selection*: la ricerca dell'architettura ottimale della rete viene affrontata mediante criteri analitici rigorosi, primo fra tutti la Generalized Cross-Validation (GCV).

Il lavoro indaga inoltre l'efficienza computazionale del modello attraverso lo studio di operazioni matriciali incrementali. Queste tecniche algebriche costituiscono il motore alla base di algoritmi di costruzione della rete come la Forward Selection e gli Orthogonal Least Squares (OLS), consentendo di selezionare dinamicamente il numero e la posizione dei centri in tempi di calcolo drasticamente ridotti. Il lavoro si conclude con una fase di validazione sperimentale: attraverso diverse simulazioni numeriche, viene dimostrato come l'accuratezza nella scelta dei centri e l'ottimizzazione dei parametri di regolarizzazione siano requisiti fondamentali per garantire prestazioni predittive ottimali su dati non noti

Indice

Sommario	i
1 Introduzione al Supervised Learning	1
1.1 Finalità dell'Elaborato	1
1.2 Vantaggi delle Reti RBF	1
1.3 Inquadramento Terminologico	2
1.4 Il Training Set e la Modellazione del Rumore	2
1.5 Regressione Parametrica vs Non Parametrica	3
2 Architettura delle Reti Radial Basis Function	4
2.1 Il Modello Lineare	4
2.2 Funzioni a Base Radiale	5
2.2.1 La Gaussiana	5
2.2.2 La Multiquadrica	5
2.3 Ottimizzazione Non Lineare dei Parametri Strutturali	6
2.4 Formulazione Matriciale	7
3 Ottimizzazione e Stima dei Pesì	8
3.1 Il Criterio dei Minimi Quadrati	8
3.2 Regularizzazione Locale e Matrice Λ	8
3.3 Derivazione dell'Equazione Normale	9
3.4 Considerazioni sulla Complessità Computazionale	10
3.5 Analisi dell'Incertezza: Varianza del Vettore dei Pesì	11
3.6 I Residui e la Matrice di Proiezione P	12
3.6.1 Rappresentazione di S e C tramite la matrice P	13
3.6.2 Gradi di Libertà Effettivi (γ)	14
4 Efficienza Computazionale e Operazioni Incrementali	16
4.1 Rimozione di un Pattern di Addestramento	16
4.2 Aggiunta di una Funzione di Base	17
5 Criteri di Validazione e Selezione del Modello	20
5.1 Leave-One-Out Cross-Validation (LOO-CV)	20
5.2 Generalized Cross-Validation (GCV)	23
6 Algoritmi di Ottimizzazione e Selezione Strutturale	25
6.1 Ottimizzazione della Regularizzazione (Ridge Regression)	25
6.1.1 La Ridge Regression	25
6.1.2 Ottimizzazione del Parametro λ	26
6.1.3 Local Ridge Regression	27
6.2 Forward Selection	30
6.2.1 Algoritmo e Criterio di Selezione	31
6.2.2 Orthogonal Least Squares (OLS)	31

Introduzione al Supervised Learning

In molte applicazioni ingegneristiche e di analisi dei dati, ci si trova a disporre esclusivamente di misurazioni sperimentali, senza conoscere la legge matematica che regola il fenomeno sottostante. L'obiettivo dell'apprendimento supervisionato (*Supervised Learning*) è proprio quello di colmare questa lacuna: invece di postulare a priori una formula, si costruisce un modello flessibile capace di "dedurre" la relazione incognita analizzando un insieme storico di input e output correlati. In questo contesto, il sistema apprende estraendo informazione da un set di esempi di addestramento, con lo scopo ultimo di generalizzare tale comportamento e prevedere correttamente l'output per sollecitazioni mai incontrate in precedenza.

1.1 Finalità dell'Elaborato

Quando si affrontano problemi di regressione complessi, la tendenza moderna è spesso quella di ricorrere ad architetture neurali fortemente non lineari. Tuttavia, dal punto di vista dell'analisi numerica, l'addestramento di tali modelli si traduce in problemi di ottimizzazione non convessi, difficili da trattare e tendenti a convergere verso minimi locali sub-ottimali.

Questo elaborato si pone come una guida teorico-pratica per affrontare l'apprendimento supervisionato attraverso le reti a funzione di base radiale (RBF), adottando invece un approccio lineare. Come verrà descritto nei capitoli successivi, mantenendo congelati i parametri non lineari della rete, è possibile ricondurre il problema della ricerca dei pesi ottimali alla risoluzione di un classico sistema lineare. In questo modo, l'obiettivo del lavoro è dimostrare come sia possibile sacrificare una minima parte della flessibilità strutturale del modello per guadagnare l'assoluta stabilità e prevedibilità offerte dagli strumenti dell'algebra lineare.

1.2 Vantaggi delle Reti RBF

Interpretare le reti RBF come modelli statistici lineari nei parametri offre vantaggi che impattano direttamente sull'implementazione algoritmica e sulle prestazioni computazionali:

- **Semplicità Matematica:** Poiché l'output è una combinazione lineare di funzioni di base, l'analisi del modello si riduce a problemi di algebra lineare, evitando le complessità teoriche dei modelli puramente non lineari.
- **Efficienza Computazionale:** L'ottimizzazione dei pesi non richiede algoritmi iterativi pesanti, ma può essere risolta tramite equazioni dirette.
- **Soluzione Ottimale Unica:** Per un set di funzioni di base fissate, è possibile derivare e risolvere un set di equazioni per i valori ottimali dei pesi, garantendo la convergenza verso un minimo globale dell'errore.

- **Operazioni Incrementali Efficienti:** La natura del modello permette di aggiungere o rimuovere funzioni di base o pattern di addestramento in modo rapido attraverso formule analitiche, senza dover riaddestrare l'intera rete da zero.
- **Trattabilità della Selezione del Modello:** Criteri come la Generalized Cross-Validation (GCV) risultano analiticamente calcolabili e meno costosi rispetto a modelli non lineari di dimensioni comparabili.

1.3 Inquadramento Terminologico

Poiché la statistica e la teoria delle reti neurali si sono evolute in parallelo, spesso condividono gli stessi concetti matematici ma utilizzano nomi diversi. Per fare chiarezza, la Tabella 1.1 riassume le corrispondenze terminologiche valide per l'intero elaborato.

Tabella 1.1: Corrispondenza terminologica tra statistica e reti neurali.

Statistica	Reti Neurali
Modello	Rete (Network)
Stima (Estimation)	Apprendimento (Learning)
Regressione	Apprendimento supervisionato
Interpolazione	Generalizzazione
Osservazioni	Set di addestramento (Training set)
Parametri	Pesi (Weights)
Variabili indipendenti	Input
Variabili dipendenti	Output
Ridge Regression	Weight Decay (Decadimento dei pesi)

1.4 Il Training Set e la Modellazione del Rumore

Nella pratica sperimentale, la nostra comprensione di un sistema reale si basa esclusivamente sui dati che riusciamo a raccogliere. Consideriamo quindi una funzione incognita $y = f(\mathbf{x})$, dove l'output scalare y dipende da un vettore di input $\mathbf{x} \in \mathbb{R}^n$. Il punto di partenza di ogni processo di apprendimento è il set di addestramento (*training set*) T , costituito da p osservazioni:

$$T = \{(\mathbf{x}_i, \hat{y}_i)\}_{i=1}^p \quad (1.1)$$

È fondamentale osservare che, in un contesto reale, le etichette $\hat{y}_i$ non rappresentano quasi mai il valore esatto della funzione, ma una sua versione perturbata da una componente stocastica incognita. Matematicamente, modelliamo questa incertezza come $\hat{y}_i = y_i + \epsilon$, dove ϵ rappresenta il rumore.

Successivamente andrebbe fatta una distinzione tra il rumore che disturba le variabili dipendenti (output) e quello che disturba gli input. Tuttavia, considerare

anche l'incertezza sui vettori $\mathbf{x}_i$ renderebbe la formulazione del problema molto pesante. Per cui si adotterà l'ipotesi semplificativa di considerare il rumore sugli input come "assorbito" all'interno della componente di errore misurata sull'output.

1.5 Regressione Parametrica vs Non Parametrica

Bisogna successivamente definire come vogliamo rappresentare matematicamente i legami tra i dati che abbiamo raccolto. Fondamentalmente, fare apprendimento significa trovare una formula o un modello che riesca a descrivere bene il comportamento del sistema che stiamo studiando.

Ma perché è così importante distinguere tra approccio parametrico e non parametrico? Il motivo è che questa scelta cambia completamente il nostro modo di lavorare. Infatti prima di iniziare l'analisi, dobbiamo chiederci quanta informazione abbiamo già sul problema: conosciamo già le leggi fisiche che lo governano o abbiamo solo un insieme di misure e dobbiamo scoprire tutto da zero?

Ecco le due strade possibili:

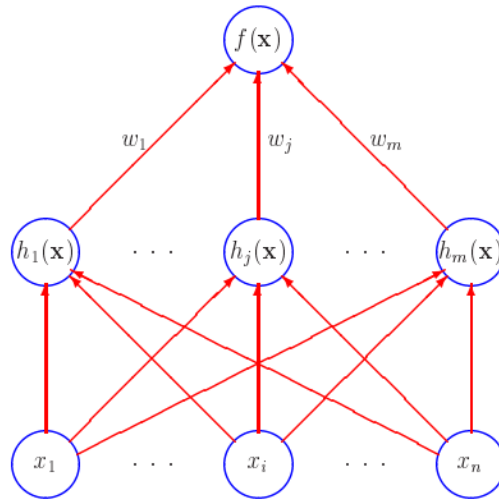
- **Regressione Parametrica:** Si usa quando sappiamo già "come è fatta" la funzione che stiamo cercando. Magari la fisica del problema ci dice che il rapporto è una retta o una parabola. In questo caso, il nostro unico compito è trovare quei pochi numeri (i parametri) che mancano per completare la formula. Questi numeri sono importanti perché hanno un significato fisico reale che possiamo interpretare.
- **Regressione Non Parametrica:** È l'approccio che usiamo per i problemi complessi, dove non abbiamo nessuna formula pronta a priori. Qui il modello deve essere molto flessibile per potersi adattare a qualsiasi forma abbiano i dati. Per farlo, usiamo moltissimi parametri (i pesi), che presi singolarmente non hanno un significato fisico, ma servono come "gradi di libertà" per permettere alla rete di seguire l'andamento dei dati nel modo più preciso possibile.

Le reti RBF appartengono a questa seconda categoria. Introdurre questa differenza serve a chiarire che con le RBF non stiamo cercando di confermare una legge fisica che già conosciamo, ma stiamo usando uno strumento capace di ricostruire da solo la forma della funzione partendo direttamente dai dati di input.

2

Architettura delle Reti Radial Basis Function

In questo capitolo analizzeremo nel dettaglio com'è fatta internamente una rete RBF. Dopo aver stabilito che il nostro obiettivo è la regressione non parametrica, dobbiamo definire i componenti fondamentali del modello: vedremo come si passa da una semplice somma di funzioni a un sistema matematico capace di apprendere dai dati.



2.1 Il Modello Lineare

L'idea alla base di una rete RBF è quella di rappresentare la funzione incognita come una combinazione lineare (una somma pesata) di m funzioni elementari, dette funzioni di base h_j :

$$f(\mathbf{x}) = \sum_{j=1}^m w_j h_j(\mathbf{x}). \quad (2.1)$$

La forza di questo approccio sta nella gestione dei gradi di libertà. In una rete RBF, la capacità di adattarsi ai dati dipende dalla scelta dei pesi w_j . Se decidiamo di mantenere fisse le funzioni di base h_j ovvero non cambiamo la loro posizione o la loro forma durante la fase di addestramento il modello rimane lineare nei parametri. Questa scelta, come già sottolineato precedentemente, ci permette di ricondurre l'apprendimento alla risoluzione di un sistema lineare diretto, garantendo stabilità numerica e un'efficienza computazionale superiore rispetto ai metodi iterativi.

2.2 Funzioni a Base Radiale

Dopo aver definito la struttura della somma, dobbiamo scegliere quali funzioni usare Nel nostro modello. Le funzioni a base radiale si distinguono perché la loro risposta non dipende dal valore assoluto dell'input, ma solo dalla distanza tra l'input $\mathbf{x}$ e un punto centrale c . In pratica, la funzione reagisce in base a quanto l'input è vicino al suo "centro".

2.2.1 La Gaussiana

La Gaussiana è la funzione più utilizzata in questo ambito. La sua espressione matematica è:

$$h(x) = \exp\left(-\frac{(x - c)^2}{r^2}\right), \quad (2.2)$$

dove c è il centro e r rappresenta il raggio (che ne controlla l'ampiezza).

La caratteristica che rende la Gaussiana così efficace è la sua "località": essa fornisce una risposta forte solo nell'intorno del centro, spegnendosi rapidamente man mano che ci si allontana. Questo permette a ogni componente della rete di specializzarsi nella descrizione di una zona specifica dello spazio dei dati.

2.2.2 La Multiquadrica

Esistono però anche alternative con un comportamento opposto, come la funzione multiquadrica:

$$h(x) = \sqrt{r^2 + (x - c)^2}. \quad (2.3)$$

A differenza della Gaussiana, questa è una funzione "globale": la sua risposta aumenta man mano che ci si allontana dal centro. Anche se matematicamente interessante, per scopi pratici si preferiscono quasi sempre le funzioni locali come la Gaussiana, poiché rendono il modello molto più stabile e meno sensibile a variazioni brusche nei dati lontani dal centro.

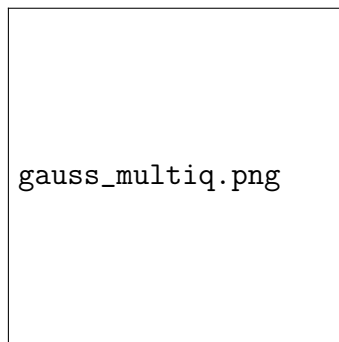


Figura 2.1: Confronto tra una funzione base locale (Gaussiana) e una globale (Multiquadrica).

2.3 Ottimizzazione Non Lineare dei Parametri Strutturali

Sebbene l'approccio lineare sia il cuore di questo elaborato, è opportuno accennare all'esistenza di una formulazione più generale in cui anche i parametri dello strato nascosto ovvero la posizione dei centri (c_j) e la loro ampiezza o raggio (r_j) vengono appresi dinamicamente dai dati. Dal punto di vista teorico, l'ottimizzazione di questi parametri garantisce una flessibilità enorme, permettendo di descrivere fenomeni complessi con modelli molto compatti, ovvero utilizzando un numero ridotto di centri m . Tuttavia, l'inserimento di tali parametri all'interno della funzione a base radiale, come nel caso della Gaussiana:

$$h_j(x) = \exp\left(-\frac{(x - c_j)^2}{r_j^2}\right) \quad (2.4)$$

distrugge la linearità del problema rispetto ai parametri strutturali. La determinazione di c_j e r_j può seguire due strade principali:

- **Approccio Ibrido/Euristico:** È la strategia più diffusa per mantenere l'efficienza computazionale. In questo caso, i centri vengono determinati tramite algoritmi di **clustering non supervisionato**, come il *k-means*.

Nota metodologica sul clustering: In termini semplici, il clustering è una tecnica che analizza la distribuzione naturale dei dati di input senza conoscerne l'etichetta (target). Utilizzare il *k-means* significa identificare i “baricentri” delle zone in cui i dati sono più densi. L'algoritmo raggruppa i dati in k gruppi e posiziona il centro c_j nel punto medio geometrico di ogni gruppo; in questo modo, i neuroni della rete non sono disposti casualmente, ma si posizionano strategicamente dove si trova la maggior parte dell'informazione. I raggi (r_j) vengono poi impostati tramite euristiche basate sulla distanza tra questi centri, garantendo una copertura omogenea dello spazio.

- **Approccio Completamente Supervisionato:** In questo caso, si utilizza la discesa del gradiente per aggiornare c_j e r_j minimizzando l'errore globale. Tuttavia questo metodo presenta criticità numeriche severe: la funzione di costo, di cui parleremo in dettaglio in seguito, diventa altamente non convessa, introducendo numerosi minimi locali che portano a una convergenza lenta o inefficiente.

Inoltre, l'ottimizzazione supervisionata dei raggi tende spesso a espanderli eccessivamente nel tentativo di ridurre l'errore globale; ciò causa la perdita della proprietà di “localizzazione” delle funzioni RBF, rendendo la rete incapace di discriminare i dettagli più fini dei dati.

Per tali ragioni, in questo lavoro manterremo il focus sul modello a centri fissi. Questa scelta permette di privilegiare la stabilità algoritmica, garantendo l'unicità del minimo globale per i pesi di output e una superiore efficienza computazionale.

2.4 Formulazione Matriciale

Quando passiamo dalla singola equazione matematica all'elaborazione pratica dell'intero *training set* (composto da p dati), ci conviene abbandonare le sommatorie e ragionare in termini vettoriali. Per farlo, costruiamo la struttura portante del nostro modello: la **Matrice di Design** $H \in \mathbb{R}^{p \times m}$:

$$H = \begin{bmatrix} h_1(\mathbf{x}_1) & \dots & h_m(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ h_1(\mathbf{x}_p) & \dots & h_m(\mathbf{x}_p) \end{bmatrix}. \quad (2.5)$$

Possiamo immaginare questa matrice come una "fotografia" completa di come reagisce la nostra rete.

Ottimizzazione e Stima dei Pesi

Dopo aver definito l'architettura della rete e costruito la matrice di design H , il problema si sposta sulla fase di addestramento. In termini matematici, questo significa trovare il vettore dei pesi $\mathbf{w}$ capace di rendere il modello il più possibile aderente alla realtà dei dati raccolti.

3.1 Il Criterio dei Minimi Quadrati

Il punto di partenza naturale per ogni problema di regressione è cercare di minimizzare lo scarto tra le previsioni della nostra rete, $f(\mathbf{x}_i)$, e i valori reali osservati, $\hat{y}_i$. Lo strumento standard per misurare questo scarto è la somma dei quadrati degli errori (SSE: *Sum of Squared Errors*):

$$S = \sum_{i=1}^p (\hat{y}_i - f(\mathbf{x}_i))^2 = \|\hat{\mathbf{y}} - H\mathbf{w}\|^2, \quad (3.1)$$

dove $\hat{\mathbf{y}}$ è il vettore dei valori reali osservati.

In un mondo ideale, dove i dati sono precisi e privi di disturbi, basterebbe minimizzare questa funzione per trovare la soluzione perfetta. Tuttavia, come abbiamo visto nel Capitolo 1, i dati sperimentali sono sempre disturbati dal rumore. In queste condizioni, cercare la minima distanza possibile dai dati può rivelarsi una strategia rischiosa e rendere il sistema estremamente instabile.

3.2 Regularizzazione Locale e Matrice Λ

Quando addestriamo una rete RBF, il nostro vero obiettivo non è passare esattamente per tutti i punti del *training set*, ma costruire un modello che sappia prevedere correttamente i valori futuri. Il pericolo principale è cadere nell'**overfitting**: un fenomeno in cui la rete, avendo troppi gradi di libertà, finisce per imparare a memoria anche le fluttuazioni casuali e gli errori di misura dei dati, perdendo di vista il trend reale della funzione. Una rete in overfitting apparirà graficamente come una curva irregolare, capace di azzerare l'errore sui dati noti ma del tutto inaffidabile su quelli nuovi.

Per evitare questo problema, dobbiamo introdurre un "freno" al modello. L'idea è quella di aggiungere alla funzione di costo (rappresentata dalla differenza tra i dati sperimentali e quelli teorici) una penalità proporzionale alla grandezza dei pesi w_j . Utilizzando dei coefficienti di penalità locali λ_j , possiamo decidere quanto irrigidire la rete in modo selettivo:

- Nelle regioni in cui i dati sono molto rumorosi, aumenteremo il valore di λ_j . Questo costringerà la rete a ignorare le fluttuazioni sospette e a seguire solo l'andamento principale dei dati.
- Dove le osservazioni sono più affidabili, manterremo λ_j basso, lasciando alla rete la flessibilità necessaria per approssimare fedelmente la funzione sottostante.

Matematicamente, questa nuova funzione di costo regolarizzata C si scrive come:

$$C = \sum_{i=1}^p \left(\hat{y}_i - \sum_{j=1}^m w_j h_j(\mathbf{x}_i) \right)^2 + \sum_{j=1}^m \lambda_j w_j^2. \quad (3.2)$$

Per gestire questa operazione in modo efficiente, raggruppiamo i coefficienti di penalità in una matrice diagonale $\Lambda \in \mathbb{R}^{m \times m}$. In questo modo, la funzione di costo finale assume la seguente forma matriciale:

$$C = \|\hat{\mathbf{y}} - H\mathbf{w}\|^2 + \mathbf{w}^T \Lambda \mathbf{w}, \quad (3.3)$$

dove:

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m) = \begin{bmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_m \end{bmatrix}. \quad (3.4)$$

3.3 Derivazione dell'Equazione Normale

Per trovare la configurazione ottimale della rete, dobbiamo individuare il vettore dei pesi $\mathbf{w}$ che minimizza la funzione di costo regolarizzata C . Per procedere analiticamente, è conveniente esplicitare la norma quadratica dell'errore riscrivendola in termini di prodotto scalare tra vettori:

$$C = (\hat{\mathbf{y}} - H\mathbf{w})^T (\hat{\mathbf{y}} - H\mathbf{w}) + \mathbf{w}^T \Lambda \mathbf{w}. \quad (3.5)$$

Sviluppando il prodotto matriciale nel primo termine della somma, otteniamo l'espressione estesa:

$$C = \hat{\mathbf{y}}^T \hat{\mathbf{y}} - \hat{\mathbf{y}}^T H\mathbf{w} - \mathbf{w}^T H^T \hat{\mathbf{y}} + \mathbf{w}^T H^T H \mathbf{w} + \mathbf{w}^T \Lambda \mathbf{w}. \quad (3.6)$$

Osserviamo che il termine $\hat{\mathbf{y}}^T H\mathbf{w}$ produce un risultato scalare. Poiché uno scalare coincide con il proprio trasposto, vale l'identità $(\hat{\mathbf{y}}^T H\mathbf{w})^T = \mathbf{w}^T H^T \hat{\mathbf{y}} = \hat{\mathbf{y}}^T H\mathbf{w}$. Sfruttando questa proprietà, possiamo raggruppare i due termini simili e raccogliere i termini quadratici in $\mathbf{w}$, ottenendo:

$$C = \hat{\mathbf{y}}^T \hat{\mathbf{y}} - 2\mathbf{w}^T H^T \hat{\mathbf{y}} + \mathbf{w}^T (H^T H + \Lambda) \mathbf{w}. \quad (3.7)$$

Trattandosi di una funzione convessa (poiché la matrice Hessiana del sistema $(H^T H + \Lambda)$ è strettamente definita positiva, grazie alla presenza dei parametri $\lambda_j > 0$ sulla diagonale di Λ), il minimo globale si ottiene annullando il gradiente rispetto al vettore dei pesi. Calcoliamo quindi la derivata vettoriale $\nabla_{\mathbf{w}} C$ e imponiamo la condizione di stazionarietà:

$$\frac{\partial C}{\partial \mathbf{w}} = -2H^T \hat{\mathbf{y}} + 2(H^T H + \Lambda)\mathbf{w} = 0. \quad (3.8)$$

Nel momento in cui imponiamo questa uguaglianza a zero, il vettore $\mathbf{w}$ cessa di essere una variabile indipendente e diventa l'incognita del nostro sistema, ovvero la

soluzione ottima che stiamo cercando. Indicheremo questa soluzione specifica con lo stimatore $\hat{\mathbf{w}}$. L'equazione diventa quindi:

$$-2H^T\hat{\mathbf{y}} + 2(H^TH + \Lambda)\hat{\mathbf{w}} = 0. \quad (3.9)$$

Semplificando il fattore comune e riordinando i termini, giungiamo al risultato fondamentale di questo approccio, noto come **Equazione Normale regolarizzata**:

$$(H^TH + \Lambda)\hat{\mathbf{w}} = H^T\hat{\mathbf{y}}. \quad (3.10)$$

Per comodità di notazione, definiamo la matrice del sistema come $\mathbf{A} = H^TH + \Lambda$. La soluzione analitica per i pesi ottimali si ottiene infine invertendo tale matrice:

$$\hat{\mathbf{w}} = \mathbf{A}^{-1}H^T\hat{\mathbf{y}}. \quad (3.11)$$

Questa equazione rappresenta il traguardo della nostra scelta di mantenere lineare il modello. Abbiamo infatti trasformato il problema originario (l'addestramento di una rete neurale) in un classico sistema lineare di m equazioni in m incognite. Questa formulazione è risolvibile in modo diretto, univoco ed efficiente sfruttando i consolidati algoritmi di decomposizione matriciale dell'algebra lineare.

È utile notare come questa formula sia una generalizzazione. Se decidessimo di azzerare la penalità ($\Lambda = \mathbf{0}$), l'equazione si ridurrebbe allo stimatore classico dei Minimi Quadrati Ordinari (OLS: *Ordinary Least Squares*): $\hat{\mathbf{w}} = (H^TH)^{-1}H^T\hat{\mathbf{y}}$. Tuttavia, come abbiamo discusso, se i dati sono rumorosi o le funzioni di base si sovrappongono troppo (rendendo la matrice H^TH mal condizionata), invertire direttamente questa matrice porta a risultati instabili. L'aggiunta della matrice diagonale Λ funge da stabilizzatore numerico, garantendo che il sistema $\mathbf{A}$ sia sempre ben condizionato e invertibile.

3.4 Considerazioni sulla Complessità Computazionale

Uno dei punti di forza di questa architettura è la capacità di elaborare moli di dati considerevoli mantenendo i tempi di calcolo sotto controllo. Se analizziamo la struttura della soluzione $\hat{\mathbf{w}} = (H^TH + \Lambda)^{-1}H^T\hat{\mathbf{y}}$, possiamo dividere il processo di addestramento in due fasi distinte per valutarne la complessità computazionale:

1. **Costruzione del sistema:** Il primo passo richiede il calcolo del prodotto H^TH . Ricordiamo che p rappresenta il numero di osservazioni (i dati di addestramento) e m il numero di centri (le funzioni di base) della rete. Moltiplicare la matrice trasposta H^T (di dimensione $m \times p$) per la matrice di design H (di dimensione $p \times m$) produce come risultato una nuova matrice quadrata di dimensione $m \times m$. Per calcolare ciascuno dei suoi m^2 elementi, l'algoritmo deve eseguire un prodotto scalare tra vettori di lunghezza p . Per cui richiede un numero di operazioni pari a $\mathcal{O}(p \cdot m^2)$. Questo significa che il tempo di costruzione cresce in modo lineare rispetto ai dati: se raccogliamo il doppio delle osservazioni (p), il tempo richiesto per eseguire i prodotti scalari semplicemente raddoppia.

2. **Risoluzione del sistema lineare:** La fase cruciale è l'inversione (o la fattorizzazione, ad esempio tramite decomposizione di Cholesky o LU) della matrice quadrata $\mathbf{A}$, che ha dimensione $m \times m$. Come noto dall'algebra lineare numerica, i metodi diretti standard per risolvere o invertire una matrice densa richiedono un numero di operazioni proporzionale ad m^3 . La complessità di questa operazione è quindi fissata a $\mathcal{O}(m^3)$.

Per cui questo secondo punto è di vitale importanza: la complessità computazionale del nostro addestramento dipende esclusivamente da quanti centri (m) abbiamo inserito nel modello, ed è del tutto indipendente dalla quantità di dati (p) analizzati. Poiché nei problemi reali di regressione non parametrica si utilizza un numero di funzioni di base molto inferiore rispetto alle osservazioni disponibili ($m \ll p$), l'addestramento risulta estremamente rapido, confermando l'efficienza delle reti RBF a centri fissi rispetto ai modelli non lineari.

3.5 Analisi dell'Incertezza: Varianza del Vettore dei Pesi

Come chiarito nel Capitolo 1, il nostro set di addestramento è composto da osservazioni $\hat{\mathbf{y}}$ che contengono un'incertezza intrinseca. Immaginiamo che esista una funzione deterministica $\mathbf{f}$ tale che:

$$\hat{\mathbf{y}} = \mathbf{f} + \epsilon, \quad (3.12)$$

Assumiamo che ϵ sia un vettore di rumore puramente casuale (gaussiano a media nulla) con una certa varianza, tale per cui la matrice di covarianza del rumore sia $\text{Cov}(\epsilon) = \sigma^2 I_p$. Poiché la funzione $\mathbf{f}$ è deterministica, l'intera incertezza delle nostre misurazioni deriva dal rumore, per cui $\text{Cov}(\hat{\mathbf{y}}) = \sigma^2 I_p$.

Il fulcro del problema è che i pesi della nostra rete ($\hat{\mathbf{w}}$) vengono calcolati algebricamente a partire dai dati misurati usando l'Equazione Normale: $\hat{\mathbf{w}} = \mathbf{A}^{-1} H^T \hat{\mathbf{y}}$ (ricordando che $\mathbf{A} = H^T H + \Lambda$). Di conseguenza, se i dati originali contengono incertezza, questa incertezza si propaga inevitabilmente ai pesi della rete, rendendo il vettore $\hat{\mathbf{w}}$ a tutti gli effetti un vettore di variabili aleatorie.

Per quantificare quanto i nostri pesi siano "incerti", possiamo calcolarne la matrice di varianza-covarianza:

$$\text{Cov}(\hat{\mathbf{w}}) = \text{Cov}(\mathbf{A}^{-1} H^T \hat{\mathbf{y}}) = (\mathbf{A}^{-1} H^T) \text{Cov}(\hat{\mathbf{y}}) (\mathbf{A}^{-1} H^T)^T, \quad (3.13)$$

dove nell'ultima uguaglianza abbiamo usato le proprietà della covarianza su costanti e il fatto che $\mathbf{A}^{-1}$ ed H^T dipendono da dati deterministici.

Sostituendo l'espressione per l'incertezza dei dati $\text{Cov}(\hat{\mathbf{y}}) = \sigma^2 I_p$ e ricordando che $\mathbf{A}$ è una matrice simmetrica (quindi $\mathbf{A}^T = \mathbf{A}$ e $(\mathbf{A}^{-1})^T = \mathbf{A}^{-1}$), il calcolo si semplifica in:

$$\text{Cov}(\hat{\mathbf{w}}) = \sigma^2 \mathbf{A}^{-1} H^T H \mathbf{A}^{-1}. \quad (3.14)$$

Per capire il reale impatto della regolarizzazione sull'incertezza, è utile esprimere questa formula isolando il termine Λ . Dalla definizione della matrice del sistema,

sappiamo che $H^T H = \mathbf{A} - \Lambda$. Sostituendo questa identità nell'equazione precedente otteniamo:

$$\text{Cov}(\hat{\mathbf{w}}) = \sigma^2 \mathbf{A}^{-1} (\mathbf{A} - \Lambda) \mathbf{A}^{-1}. \quad (3.15)$$

Sviluppando il prodotto matriciale e ricordando che il prodotto di una matrice per la sua inversa dà l'identità ($\mathbf{A}^{-1} \mathbf{A} = I$), giungiamo alla forma finale della matrice di covarianza dei pesi:

$$\text{Cov}(\hat{\mathbf{w}}) = \sigma^2 (\mathbf{A}^{-1} - \mathbf{A}^{-1} \Lambda \mathbf{A}^{-1}). \quad (3.16)$$

Questa equazione è fondamentale perché rappresenta la dimostrazione del **compromesso bias-varianza**: il termine "bias" (distorsione) rappresenta l'errore sistematico che commettiamo quando costringiamo un modello a essere meno flessibile, mentre la "varianza" indica quanto le stime del modello risultino instabili a causa del rumore presente nel set di addestramento. Il segno meno davanti all'ultimo termine ($-\mathbf{A}^{-1} \Lambda \mathbf{A}^{-1}$) ci dice chiaramente che l'introduzione dei parametri di penalità Λ sottrae incertezza al sistema. In sintesi, costringendo il modello a non seguire perfettamente i dati (introducendo un *bias*), otteniamo un abbattimento drastico della varianza e dell'instabilità dei pesi ($\text{Cov}(\hat{\mathbf{w}})$), garantendo predizioni più affidabili su dati futuri.

3.6 I Residui e la Matrice di Proiezione P

Una volta determinata la stima ottimale dei pesi $\hat{\mathbf{w}}$, l'obiettivo primario è quantificare l'errore di approssimazione sui dati di addestramento. Definiamo il vettore dei residui $\mathbf{e}$ come la differenza tra le osservazioni rumorose $\hat{\mathbf{y}}$ e le predizioni generate dal modello (il vettore $H\hat{\mathbf{w}}$):

$$\mathbf{e} = \hat{\mathbf{y}} - H\hat{\mathbf{w}}. \quad (3.17)$$

Sostituendo l'espressione analitica dello stimatore $\hat{\mathbf{w}} = \mathbf{A}^{-1} H^T \hat{\mathbf{y}}$, otteniamo una relazione diretta tra le osservazioni e i rispettivi errori residui:

$$\mathbf{e} = \hat{\mathbf{y}} - H(\mathbf{A}^{-1} H^T \hat{\mathbf{y}}) = (I_p - H\mathbf{A}^{-1} H^T) \hat{\mathbf{y}}, \quad (3.18)$$

dove I_p è la matrice identità di dimensione $p \times p$.

Definiamo l'operatore $P \in \mathbb{R}^{p \times p}$ come la **matrice di proiezione dei residui**:

$$P = I_p - H\mathbf{A}^{-1} H^T. \quad (3.19)$$

In questa forma, la matrice simmetrica P agisce come un operatore lineare. In particolare agisce sul vettore delle osservazioni $\hat{\mathbf{y}}$ contiene al suo interno sia i dati che il modello riesce a catturare, sia l'errore che non riesce a spiegare. L'operazione di prodotto matrice-vettore $P\hat{\mathbf{y}}$ sottrae istantaneamente dai dati tutta la parte di informazione spiegabile dall'architettura della rete, estraendo direttamente il residuo. Da qui l'identità fondamentale $\mathbf{e} = P\hat{\mathbf{y}}$.

Ma la reale importanza di definire questa matrice risiede nell'isolamento dell'architettura dai parametri. La matrice P è costruita esclusivamente a partire dalla matrice di design H e dai coefficienti di regolarizzazione Λ ; è quindi completamente svincolata dal vettore dei pesi $\hat{\mathbf{w}}$.

Come dimostreremo nei passaggi successivi, la matrice P ci permette di calcolare l'Errore Quadratico (S) e il Costo Regularizzato (C) della rete lavorando direttamente sui dati di partenza, senza alcun bisogno di esplicitare i pesi ottimali. Questa indipendenza ci permetterà di valutare e confrontare la validità di architetture diverse in modo diretto.

3.6.1 Rappresentazione di S e C tramite la matrice P

Indichiamo con S la **Somma degli Errori al Quadrato** (*Sum of Squared Errors*, SSE), definita come il prodotto scalare del vettore dei residui con sé stesso. Sfruttando la simmetria di P (ovvero $P = P^T$), possiamo riscrivere S esclusivamente in funzione delle osservazioni:

$$S = \mathbf{e}^T \mathbf{e} = (P\hat{\mathbf{y}})^T (P\hat{\mathbf{y}}) = \hat{\mathbf{y}}^T P^T P \hat{\mathbf{y}} = \hat{\mathbf{y}}^T P^2 \hat{\mathbf{y}}. \quad (3.20)$$

Inoltre la matrice P permette di esprimere il valore minimo raggiunto dalla **Funzione di Costo regularizzata** C . Partiamo dalla definizione della funzione di costo valutata in corrispondenza dei pesi ottimali $\hat{\mathbf{w}}$:

$$C = (\hat{\mathbf{y}} - H\hat{\mathbf{w}})^T (\hat{\mathbf{y}} - H\hat{\mathbf{w}}) + \hat{\mathbf{w}}^T \Lambda \hat{\mathbf{w}}. \quad (3.21)$$

Definendo il vettore dei residui $\mathbf{e} = \hat{\mathbf{y}} - H\hat{\mathbf{w}}$, possiamo riscrivere la funzione di costo come:

$$C = \mathbf{e}^T \mathbf{e} + \hat{\mathbf{w}}^T \Lambda \hat{\mathbf{w}}. \quad (3.22)$$

Per semplificare il secondo termine, sfruttiamo l'Equazione Normale ($H^T H + \Lambda \hat{\mathbf{w}} = H^T \hat{\mathbf{y}}$). Isolando il termine $\Lambda \hat{\mathbf{w}}$ otteniamo:

$$\Lambda \hat{\mathbf{w}} = H^T \hat{\mathbf{y}} - H^T H \hat{\mathbf{w}} = H^T (\hat{\mathbf{y}} - H\hat{\mathbf{w}}) = H^T \mathbf{e}. \quad (3.23)$$

Sostituendo questa relazione nel termine di penalità della funzione di costo:

$$\hat{\mathbf{w}}^T (\Lambda \hat{\mathbf{w}}) = \hat{\mathbf{w}}^T H^T \mathbf{e} = (H\hat{\mathbf{w}})^T \mathbf{e}. \quad (3.24)$$

Ricomponendo l'equazione per C e raccogliendo il vettore $\mathbf{e}$ a destra, otteniamo una forma notevolmente compatta:

$$C = \mathbf{e}^T \mathbf{e} + (H\hat{\mathbf{w}})^T \mathbf{e} = (\mathbf{e} + H\hat{\mathbf{w}})^T \mathbf{e}. \quad (3.25)$$

Poiché per definizione $\mathbf{e} + H\hat{\mathbf{w}} = \hat{\mathbf{y}}$, la funzione di costo si riduce al semplice prodotto scalare tra i dati osservati e i residui:

$$C = \hat{\mathbf{y}}^T \mathbf{e}. \quad (3.26)$$

Infine, ricordando la proprietà della matrice di proiezione per cui $\mathbf{e} = P\hat{\mathbf{y}}$, giungiamo alla forma finale:

$$C = \hat{\mathbf{y}}^T P \hat{\mathbf{y}}. \quad (3.27)$$

Queste due equazioni dimostrano che la matrice P racchiude in sé l'intera informazione sulle prestazioni della rete: P^2 governa l'aderenza ai dati, mentre P governa il compromesso tra errore e regolarizzazione.

3.6.2 Gradi di Libertà Effettivi (γ)

La struttura matriciale appena derivata ci permette di definire un concetto cardine: il **numero effettivo di parametri** del modello, indicato con γ . Il numero di dimensioni (o gradi di libertà) impiegati da uno stimatore lineare per adattarsi ai dati è misurato dalla traccia della sua matrice di proiezione.

Poiché la nostra predizione è $H\hat{\mathbf{w}} = H\mathbf{A}^{-1}H^T\hat{\mathbf{y}}$ (ottenuta moltiplicando per H a destra e sinistra nell'equazione normale), la traccia di questa matrice definisce i parametri effettivi del modello:

$$\gamma = \text{tr}(H\mathbf{A}^{-1}H^T). \quad (3.28)$$

Di conseguenza, i gradi di libertà residui del sistema (quelli non utilizzati dal modello) sono dati dalla traccia della matrice dei residui P :

$$\text{tr}(P) = \text{tr}(I_p) - \text{tr}(H\mathbf{A}^{-1}H^T) = p - \gamma. \quad (3.29)$$

Per comprendere il significato pratico di γ , è utile confrontarlo con il numero fisico di funzioni a base radiale della rete, indicato con m . In assenza di regolarizzazione ($\Lambda = \mathbf{0}$), lo stimatore $\hat{\mathbf{w}}$ coincide con i minimi quadrati classici; in questo caso limite, la rete utilizza tutta la sua flessibilità strutturale, ottenendo $\gamma = m$ e gradi di libertà residui pari a $p - m$.

Tuttavia, l'introduzione della matrice di penalità Λ vincola la grandezza dei pesi, limitando la loro libertà di fluttuare per adattarsi al rumore. Di conseguenza, il modello sfrutta meno gradi di libertà rispetto a quelli fisicamente a disposizione, portando alla disuguaglianza fondamentale:

$$0 \leq \gamma \leq m. \quad (3.30)$$

Per dimostrare analiticamente questa contrazione dei gradi di libertà, possiamo sfruttare la proprietà ciclica della traccia, per cui $\text{tr}(AB) = \text{tr}(BA)$, e riscrivere il parametro γ come:

$$\gamma = \text{tr}(H\mathbf{A}^{-1}H^T) = \text{tr}(\mathbf{A}^{-1}H^TH). \quad (3.31)$$

Assumendo per semplicità un parametro di regolarizzazione scalare globale, la matrice del sistema è $\mathbf{A} = H^TH + \lambda I_m$. Possiamo quindi esplicitare il termine relativo alla matrice di design come $H^TH = \mathbf{A} - \lambda I_m$. Sostituendo questa equivalenza all'interno della traccia, lo sviluppo algebrico è immediato:

$$\gamma = \text{tr}(\mathbf{A}^{-1}(\mathbf{A} - \lambda I_m)) = \text{tr}(I_m - \lambda \mathbf{A}^{-1}) = m - \lambda \text{tr}(\mathbf{A}^{-1}), \quad (3.32)$$

dove abbiamo sfruttato la linearità dell'operatore traccia e il fatto che la traccia della matrice identità di dimensione m è pari a m .

A questo punto, indichiamo con d_i (per $i = 1, \dots, m$) gli autovalori della matrice simmetrica H^TH . Per le proprietà dell'algebra lineare, gli autovalori della matrice inversa $\mathbf{A}^{-1} = (H^TH + \lambda I_m)^{-1}$ sono pari a $\frac{1}{\lambda + d_i}$.

Ricordando che la traccia di una matrice coincide con la somma dei suoi autovalori, possiamo sostituire questo termine nell'equazione precedente:

$$\gamma = m - \lambda \text{tr}(\mathbf{A}^{-1}) = m - \lambda \sum_{i=1}^m \frac{1}{\lambda + d_i} = \sum_{i=1}^m \left(1 - \frac{\lambda}{\lambda + d_i}\right) = \sum_{i=1}^m \frac{d_i}{\lambda + d_i}. \quad (3.33)$$

Questa sommatoria rivela in modo analitico l'effetto della regolarizzazione:

- Se $\lambda \rightarrow 0$, ogni termine della sommatoria tende a $d_i/d_i = 1$, restituendo $\gamma = m$ (i gradi di libertà fisici massimi).
- Se $\lambda > 0$, ogni parametro contribuisce alla complessità del modello con una frazione strettamente minore di 1, riducendo l'effettiva flessibilità totale della rete.

Questo parametro γ fornisce una misura dell'effettiva complessità impiegata dalla rete. In termini pratici, γ ci dice quanta della flessibilità originaria (gli m centri) stiamo realmente utilizzando per interpolare il *training set*, al netto del "freno" imposto dalla regolarizzazione. Il suo calcolo costituisce il fondamento teorico per un criterio di selezione automatica dell'architettura che verrà analizzato nei capitoli successivi: la *Generalized Cross-Validation* (GCV).

Efficienza Computazionale e Operazioni Incrementali

L'implementazione pratica della selezione del modello non si esaurisce in un singolo addestramento. Al contrario, richiede di generare, valutare e confrontare un numero elevato di architetture candidate prima di scegliere la migliore. Come vedremo nei prossimi capitoli, algoritmi come la *Cross-Validation* o la *Forward Selection* procedono in modo iterativo: valutano le prestazioni del modello escludendo ciclicamente un dato di addestramento alla volta, oppure ne espandono l'architettura aggiungendo una nuova funzione di base alla volta.

Per ogni singola configurazione generata, la rete richiede il calcolo diretto dei pesi ottimali e della matrice di proiezione P . Entrambe queste operazioni necessitano dell'inversione della matrice del sistema $\mathbf{A} = H^T H + \Lambda$. Come abbiamo stabilito in precedenza, l'inversione matriciale ha una complessità computazionale cubica, pari a $\mathcal{O}(m^3)$. Se dovessimo riaddestrare la rete da zero (ovvero ricostruire e invertire $\mathbf{A}$) ad ogni singola iterazione di ricerca, l'esecuzione richiederebbe tempi di calcolo eccessivi, rendendo l'algoritmo proibitivo nella pratica.

È qui che l'impostazione lineare della nostra rete RBF mostra uno dei suoi maggiori vantaggi. Poiché le modifiche al modello tra un'iterazione e l'altra sono minime (nello specifico, togliamo una riga dalla matrice H durante la *Cross-Validation*, o le aggiungiamo una colonna durante la *Forward Selection*), possiamo sfruttare le potenti formule delle **operazioni incrementali**. Queste tecniche dell'algebra matriciale ci permettono di ricavare la nuova matrice inversa aggiornando semplicemente quella calcolata al passo precedente. L'operazione di aggiornamento richiede un tempo solo quadratico $\mathcal{O}(m^2)$, garantendo un risparmio computazionale enorme.

Ai fini della selezione automatica del modello, faremo affidamento su due specifiche operazioni incrementali, di cui presentiamo i risultati analitici e pratici fondamentali.

4.1 Rimozione di un Pattern di Addestramento

Nello studio del supervised learning e nella validazione dei modelli, l'operazione più frequente consiste nel determinare come cambiano le prestazioni e i parametri della rete se la priviamo di un singolo dato di addestramento.

Sia $\mathbf{A}$ la matrice del sistema originale, calcolata sull'intero dataset di p campioni. Se decidiamo di escludere l' i -esimo campione, stiamo rimuovendo l' i -esima riga dalla matrice di design H . Indichiamo questo vettore riga con $\mathbf{h}_i^T$. Dal punto di vista algebrico, eliminare un singolo dato significa sottrarre alla matrice $\mathbf{A}$ esclusivamente il suo specifico contributo.

Questo contributo è descritto dal prodotto esterno $\mathbf{h}_i \mathbf{h}_i^T$. Moltiplicando un vettore colonna per un vettore riga, otteniamo una matrice quadrata della stessa dimensione di $\mathbf{A}$ (ovvero $m \times m$). Tuttavia, poiché tutte le colonne di questa nuova matrice sono semplici multipli del singolo vettore $\mathbf{h}_i$, essa possiede rango 1.

Per questo motivo, si afferma che la rimozione di un dato provoca una perturbazione di rango 1 sulla matrice originaria.

La nuova matrice del sistema, calcolata sui restanti $p-1$ dati, risulta semplicemente:

$$\mathbf{A}_{(-i)} = \mathbf{A} - \mathbf{h}_i \mathbf{h}_i^T. \quad (4.1)$$

Invertire fisicamente questa nuova matrice per ricalcolare i pesi costerebbe $\mathcal{O}(m^3)$ operazioni. Tuttavia, la formula di **Sherman-Morrison** ci fornisce una scorciatoia per aggiornare l'inversa a seguito di perturbazioni vettoriali. Grazie ad essa, la nuova inversa $\mathbf{A}_{(-i)}^{-1}$ può essere calcolata analiticamente a partire dall'inversa originaria $\mathbf{A}^{-1}$, che è già nota:

$$\mathbf{A}_{(-i)}^{-1} = \mathbf{A}^{-1} + \frac{\mathbf{A}^{-1} \mathbf{h}_i \mathbf{h}_i^T \mathbf{A}^{-1}}{1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i}. \quad (4.2)$$

Il vantaggio di questa formulazione è prettamente computazionale: risolvere il membro di destra richiede esclusivamente moltiplicazioni tra matrici e vettori. Questo abbatte la complessità di aggiornamento a $\mathcal{O}(m^2)$.

Oltre al risparmio di calcolo, questa equazione rivela una proprietà algebrica di eccezionale importanza per le nostre analisi future. Osservando il denominatore del termine di aggiornamento, notiamo che esso coincide esattamente con l'elemento sulla diagonale principale della matrice di proiezione P del modello completo, relativa all' i -esimo dato:

$$P_{ii} = 1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i. \quad (4.3)$$

Come vedremo nel prossimo capitolo, essa costituirà lo strumento fondamentale che ci permetterà di derivare le formule della *Cross-Validation* in modo analitico e istantaneo, eludendo del tutto la necessità di riaddestrare la rete a ogni iterazione.

4.2 Aggiunta di una Funzione di Base

In contrapposizione alla rimozione dei dati, la fase costruttiva della rete richiede l'operazione opposta: l'espansione dell'architettura tramite l'inserimento di una nuova funzione a base radiale.

Supponiamo di disporre di un modello parziale, composto da m centri, e di voler valutare l'impatto dell'aggiunta di un'ulteriore $(m+1)$ -esima funzione di base. Dal punto di vista strutturale, l'inserimento di questo nuovo centro espande la matrice di design H_m (aggiungendo un nuovo vettore colonna, che indicheremo con $\mathbf{h}_{m+1}$). Di conseguenza, la nuova matrice del sistema $\mathbf{A}_{m+1} = H_{m+1}^T H_{m+1} + \Lambda_{m+1}$ assume una ben precisa configurazione blocchi:

$$\mathbf{A}_{m+1} = \begin{bmatrix} \mathbf{A}_m & \mathbf{b} \\ \mathbf{b}^T & c \end{bmatrix}, \quad (4.4)$$

dove $\mathbf{A}_m$ è la vecchia matrice del sistema, $\mathbf{b} = H_m^T \mathbf{h}_{m+1}$ è un vettore colonna che contiene i prodotti scalari (e quindi misura la sovrapposizione) tra le funzioni di base già presenti e il nuovo centro candidato, mentre lo scalare $c = \mathbf{h}_{m+1}^T \mathbf{h}_{m+1} + \lambda_{m+1}$ rappresenta la norma al quadrato del nuovo vettore regolarizzata dalla sua penalità.

Invece di invertire manualmente questa nuova matrice di dimensione $(m+1) \times (m+1)$ sopportando il suo costo cubico, possiamo diminuirne il suo costo utilizzando il lemma di inversione per matrici a blocchi. Per applicarlo, dobbiamo innanzitutto

calcolare uno scalare Δ , noto nell'algebra lineare come il *complemento di Schur* del blocco $\mathbf{A}_m$:

$$\Delta = c - \mathbf{b}^T \mathbf{A}_m^{-1} \mathbf{b}. \quad (4.5)$$

Sostituendo le definizioni di c e $\mathbf{b}$ all'interno di Δ , emerge un'importante proprietà algebrica:

$$\begin{aligned} \Delta &= \mathbf{h}_{m+1}^T \mathbf{h}_{m+1} + \lambda_{m+1} - (\mathbf{h}_{m+1}^T H_m) \mathbf{A}_m^{-1} (H_m^T \mathbf{h}_{m+1}) \\ &= \lambda_{m+1} + \mathbf{h}_{m+1}^T (I_p - H_m \mathbf{A}_m^{-1} H_m^T) \mathbf{h}_{m+1} \\ &= \lambda_{m+1} + \mathbf{h}_{m+1}^T P_m \mathbf{h}_{m+1}. \end{aligned} \quad (4.6)$$

Da cui si evince che il complemento di Schur Δ si rivela essere una funzione diretta della matrice di proiezione dei residui P_m calcolata al passo precedente.

Conoscendo Δ , il lemma ci permette di calcolare l'inversa della nuova matrice a blocchi in forma analitica esatta:

$$\mathbf{A}_{m+1}^{-1} = \begin{bmatrix} \mathbf{A}_m^{-1} + \frac{1}{\Delta} \mathbf{A}_m^{-1} \mathbf{b} \mathbf{b}^T \mathbf{A}_m^{-1} & -\frac{1}{\Delta} \mathbf{A}_m^{-1} \mathbf{b} \\ -\frac{1}{\Delta} \mathbf{b}^T \mathbf{A}_m^{-1} & \frac{1}{\Delta} \end{bmatrix}. \quad (4.7)$$

Questa formulazione incrementale permette non solo di aggiornare i pesi del modello in un tempo puramente quadratico $\mathcal{O}(m^2)$, ma ci fornisce lo strumento per derivare un aggiornamento diretto per la nuova matrice di proiezione P_{m+1} senza doverla ricalcolare da capo:

$$P_{m+1} = P_m - \frac{P_m \mathbf{h}_{m+1} \mathbf{h}_{m+1}^T P_m}{\Delta}. \quad (4.8)$$

Quest'ultima equazione rappresenta il vero motore degli algoritmi di costruzione delle reti neurali (come la *Forward Selection*). Il suo enorme potere risiede nel permettere di quantificare istantaneamente e a priori l'esatta riduzione della Somma degli Errori al Quadrato (ΔS) che si otterrebbe se inglobassimo il nuovo centro.

Ricordando infatti la formula sviluppata nel Capitolo 3 per cui l'errore del modello con m centri equivale a $S_m = \hat{\mathbf{y}}^T P_m \hat{\mathbf{y}}$, la riduzione garantita dall'aggiunta del nuovo nodo è la differenza algebrica:

$$S_m - S_{m+1} = \hat{\mathbf{y}}^T P_m \hat{\mathbf{y}} - \hat{\mathbf{y}}^T P_{m+1} \hat{\mathbf{y}} = \hat{\mathbf{y}}^T (P_m - P_{m+1}) \hat{\mathbf{y}}. \quad (4.9)$$

Sostituendo l'espressione incrementale appena ricavata per la differenza delle matrici di proiezione, otteniamo:

$$S_m - S_{m+1} = \hat{\mathbf{y}}^T \left(\frac{P_m \mathbf{h}_{m+1} \mathbf{h}_{m+1}^T P_m}{\Delta} \right) \hat{\mathbf{y}} = \frac{(\hat{\mathbf{y}}^T P_m \mathbf{h}_{m+1})(\mathbf{h}_{m+1}^T P_m \hat{\mathbf{y}})}{\Delta}. \quad (4.10)$$

A questo punto, sfruttando la simmetria della matrice di proiezione ($P_m = P_m^T$), notiamo che il blocco di destra al numeratore, ovvero $(\mathbf{h}_{m+1}^T P_m \hat{\mathbf{y}})$, coincide esattamente con la trasposta dello scalare a sinistra $(\hat{\mathbf{y}}^T P_m \mathbf{h}_{m+1})$. Poiché la trasposta di uno scalare è lo scalare stesso, il numeratore si compatta e giungiamo così alla formula risolutiva:

$$S_m - S_{m+1} = \frac{(\hat{\mathbf{y}}^T P_m \mathbf{h}_{m+1})^2}{\Delta}. \quad (4.11)$$

Questa formula racchiude il motivo dell'efficienza dei modelli lineari nei parametri: grazie ad essa, un algoritmo può testare migliaia di centri spaziali candidati in una frazione di secondo. Limitandosi a calcolare semplici prodotti matrice-vettore (senza mai invertire matrici), la rete è in grado di prevedere quale funzione di base apporterà il maggior beneficio e di selezionarla iterativamente per massimizzare la riduzione dell'errore.

Criteri di Validazione e Selezione del Modello

Nei capitoli precedenti abbiamo analizzato in dettaglio il processo di ottimizzazione dei pesi $\hat{\mathbf{w}}$ per una rete RBF, muovendoci sotto l'assunzione che la struttura del modello (il numero di funzioni e i parametri di penalità) fosse già stata definita a monte. Tuttavia l'efficacia di una rete neurale dipende criticamente da due scelte strutturali fondamentali: quanti centri inserire nell'architettura e come tarare i parametri di regolarizzazione λ_j .

La ricerca dell'architettura ottimale è un delicato esercizio dominato dal compromesso bias-varianza. Un modello troppo semplice (con pochi centri o una regolarizzazione eccessivamente rigida) non avrà sufficiente flessibilità per catturare la dinamica dei dati, producendo stime molto stabili ma distorte (*underfitting*). Al contrario, un modello troppo complesso o poco vincolato sfrutterà i suoi gradi di libertà per interpolare alla perfezione anche il rumore e gli errori di misura presenti nel training set, perdendo del tutto la capacità di generalizzare l'andamento reale su nuovi campioni futuri (*overfitting*).

L'obiettivo in questo capitolo è individuare un **criterio di selezione quantitativo**, ovvero una formula matematica che, valutando esclusivamente i dati che abbiamo già a disposizione, sia in grado di stimare le prestazioni future della rete, penalizzandone l'eccessiva complessità matematica.

5.1 Leave-One-Out Cross-Validation (LOO-CV)

Per valutare in modo oggettivo come un modello si comporterà nel mondo reale, la logica impone di testarlo su campioni che non sono mai stati usati per addestrarlo. Tuttavia, raccogliere nuovi dati è spesso costoso o non è possibile. Per risolvere questo paradosso e massimizzare l'uso delle osservazioni già a disposizione, in statistica si ricorre alla tecnica della validazione incrociata (*Cross-Validation*).

L'idea fondamentale di questo approccio è partizionare il dataset in due sottoinsiemi mutuamente esclusivi: un *training set* per l'ottimizzazione dei parametri e un *validation set* (o set di test) per misurare l'errore predittivo "al buio". Per evitare che la valutazione dipenda da una singola partizione sfortunata (magari inserendo tutti i dati rumorosi nel set di test), la procedura prevede di ripetere l'addestramento più volte, ruotando i campioni tra i due sottoinsiemi e calcolando la media finale degli errori ottenuti.

Il caso limite e più rigoroso di questa tecnica si ottiene quando la dimensione del *validation set* viene ridotta al minimo indispensabile: un singolo campione. Questo approccio prende il nome di **Leave-One-Out Cross-Validation** (LOO-CV).

Dato un dataset di p osservazioni, la LOO-CV prevede di addestrare il modello p volte consecutive. A ogni iterazione i , una singola osservazione viene "nascosta" all'algoritmo di apprendimento per fungere da prova. Sia $f_i(\mathbf{x}_i)$ la predizione del modello per l' i -esimo pattern calcolata dopo aver addestrato la rete sui restanti $p - 1$ dati. L'obiettivo della LOO-CV è stimare la varianza dell'errore di generalizzazione

($\hat{\sigma}_{LOO}^2$) calcolando la media di questi p errori quadratici predittivi:

$$\hat{\sigma}_{LOO}^2 = \frac{1}{p} \sum_{i=1}^p (\hat{y}_i - f_i(\mathbf{x}_i))^2. \quad (5.1)$$

Da un punto di vista operativo, l'implementazione di questo algoritmo risulterebbe eccessivamente costosa: richiederebbe la costruzione e l'inversione di p sistemi lineari distinti. Poiché l'inversione di una matrice ha un costo computazionale di ordine cubico $\mathcal{O}(m^3)$, iterare questo processo p volte renderebbe i tempi di addestramento molto lenti. Tuttavia, sfruttando i risultati sulle operazioni incrementali ricavati nel capitolo precedente, possiamo eludere del tutto l'iterazione e ricavare una soluzione analitica istantanea.

Siano $H_{(-i)}$ e $\mathbf{A}_{(-i)}^{-1}$ rispettivamente la matrice di design e la matrice della varianza del sistema ridotto (ovvero privato dell' i -esima riga), e sia $\hat{\mathbf{y}}_{(-i)}$ il vettore dei target accorciato. Il vettore dei pesi ottimizzato su questo sottoinsieme è $\hat{\mathbf{w}}_{(-i)} = \mathbf{A}_{(-i)}^{-1} H_{(-i)}^T \hat{\mathbf{y}}_{(-i)}$. L'errore predittivo calcolato sull'unico dato escluso si esprime pertanto come:

$$\hat{y}_i - f_i(\mathbf{x}_i) = \hat{y}_i - \mathbf{h}_i^T \hat{\mathbf{w}}_{(-i)} = \hat{y}_i - \mathbf{h}_i^T \mathbf{A}_{(-i)}^{-1} H_{(-i)}^T \hat{\mathbf{y}}_{(-i)}. \quad (5.2)$$

Per esprimere questa quantità unicamente in funzione delle matrici del modello completo, notiamo innanzitutto che la rimozione dell' i -esima riga implica l'identità $H_{(-i)}^T \hat{\mathbf{y}}_{(-i)} = H^T \hat{\mathbf{y}} - \mathbf{h}_i \hat{y}_i$. Inoltre, richiamando l'identità di Sherman-Morrison discussa in precedenza, l'aggiornamento dell'inversa ci restituisce la seguente equivalenza:

$$\mathbf{A}_{(-i)}^{-1} \mathbf{h}_i = \frac{\mathbf{A}^{-1} \mathbf{h}_i}{1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i}. \quad (5.3)$$

Sostituendo queste due relazioni nel termine di predizione $\mathbf{h}_i^T \hat{\mathbf{w}}_{(-i)}$, possiamo sfruttare la seguente proprietà: essendo la predizione un singolo valore scalare ($\hat{y}_i - f_i(\mathbf{x}_i)$), essa coincide esattamente con la propria trasposta. Invertendo l'ordine dei fattori e ricordando che la matrice di covarianza $\mathbf{A}_{(-i)}^{-1}$ è simmetrica (pertanto coincide con la sua trasposta), lo sviluppo algebrico diventa:

$$\begin{aligned} \mathbf{h}_i^T \mathbf{A}_{(-i)}^{-1} H_{(-i)}^T \hat{\mathbf{y}}_{(-i)} &= \left(\mathbf{h}_i^T \mathbf{A}_{(-i)}^{-1} H_{(-i)}^T \hat{\mathbf{y}}_{(-i)} \right)^T \\ &= (H_{(-i)}^T \hat{\mathbf{y}}_{(-i)})^T \mathbf{A}_{(-i)}^{-1} \mathbf{h}_i \\ &= (H^T \hat{\mathbf{y}} - \mathbf{h}_i \hat{y}_i)^T \left(\frac{\mathbf{A}^{-1} \mathbf{h}_i}{1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i} \right) \\ &= \frac{\hat{\mathbf{y}}^T H \mathbf{A}^{-1} \mathbf{h}_i - \hat{y}_i \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i}{1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i}. \end{aligned} \quad (5.4)$$

Sottraendo ora questa quantità dal valore misurato $\hat{y}_i$, molti termini si elidono. L'errore commesso sul singolo dato escluso si riduce a:

$$\begin{aligned}
\hat{y}_i - f_i(\mathbf{x}_i) &= \hat{y}_i - \frac{\hat{\mathbf{y}}^T H \mathbf{A}^{-1} \mathbf{h}_i - \hat{y}_i \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i}{1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i} \\
&= \frac{\hat{y}_i - \hat{y}_i \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i - \hat{\mathbf{y}}^T H \mathbf{A}^{-1} \mathbf{h}_i + \hat{y}_i \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i}{1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i} \\
&= \frac{\hat{y}_i - \hat{\mathbf{y}}^T H \mathbf{A}^{-1} \mathbf{h}_i}{1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i}. \tag{5.5}
\end{aligned}$$

Osservando il risultato finale, emerge una corrispondenza con la matrice di proiezione dei residui del modello completo, definita come $P = I_p - H \mathbf{A}^{-1} H^T$. Lo si può dimostrare formalmente esplicitando i termini della matrice P :

- **Il denominatore (P_{ii}):** L' i -esimo elemento sulla diagonale principale di P si ottiene sottraendo a 1 (l'elemento diagonale della matrice identità I_p) l' i -esimo elemento diagonale della matrice $H \mathbf{A}^{-1} H^T$. Poiché la riga i -esima di H è il vettore $\mathbf{h}_i^T$ e la colonna i -esima di H^T è il vettore $\mathbf{h}_i$, il loro prodotto interno restituisce esattamente la forma quadratica $\mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i$. Pertanto, il denominatore della nostra frazione coincide per definizione con $P_{ii} = 1 - \mathbf{h}_i^T \mathbf{A}^{-1} \mathbf{h}_i$.
- **Il numeratore ($[P\hat{\mathbf{y}}]_i$):** Il vettore colonna dei residui sull'intero dataset è calcolato come $\mathbf{e} = P\hat{\mathbf{y}} = \hat{\mathbf{y}} - H \mathbf{A}^{-1} H^T \hat{\mathbf{y}}$. Se estraiamo esclusivamente l' i -esima riga di questo vettore, isoliamo la singola componente $\hat{y}_i - \mathbf{h}_i^T \mathbf{A}^{-1} H^T \hat{\mathbf{y}}$. Poiché questo termine rappresenta il residuo (ed è quindi uno scalare), possiamo trasporne il secondo addendo senza alterarne il valore: $(\mathbf{h}_i^T \mathbf{A}^{-1} H^T \hat{\mathbf{y}})^T = \hat{\mathbf{y}}^T H \mathbf{A}^{-1} \mathbf{h}_i$. Il risultato, $\hat{y}_i - \hat{\mathbf{y}}^T H \mathbf{A}^{-1} \mathbf{h}_i$, è identico al numeratore dell'equazione.

Questo permette di compattare l'insieme dei p errori di predizione incrociata in un unico vettore colonna, sfruttando una matrice diagonale, indicata con $\text{diag}(P)$, che contiene unicamente gli elementi P_{ii} :

$$\begin{bmatrix} \hat{y}_1 - f_1(\mathbf{x}_1) \\ \vdots \\ \hat{y}_p - f_p(\mathbf{x}_p) \end{bmatrix} = (\text{diag}(P))^{-1} P \hat{\mathbf{y}}. \tag{5.6}$$

Poiché lo stimatore $\hat{\sigma}_{LOO}^2$ non è altro che la media dei quadrati di questi errori, allora questo, equivale al prodotto scalare di questo nuovo vettore per se stesso, diviso per p . Si giunge così alla formulazione finale esatta:

$$\hat{\sigma}_{LOO}^2 = \frac{1}{p} \hat{\mathbf{y}}^T P (\text{diag}(P))^{-2} P \hat{\mathbf{y}}. \tag{5.7}$$

Questo è un ottimo risultato dal punto di vista teorico, poiché azzerà i tempi di esecuzione, richiedendo l'addestramento di un solo modello. Tuttavia, dal punto di vista dell'implementazione numerica, l'impiego diretto della matrice $\text{diag}(P)$ lo

rende fortemente instabile. Infatti se, per una particolare configurazione dei dati, un elemento diagonale P_{ii} assumesse un valore prossimo allo zero, il termine al quadrato al denominatore divergerebbe, causando un picco anomalo nella stima dell'errore che comprometterebbe l'intera selezione dell'architettura. È per ovviare a questa criticità numerica che diviene necessaria l'introduzione di un criterio generalizzato.

5.2 Generalized Cross-Validation (GCV)

Come abbiamo già descritto, il calcolo analitico della validazione Leave-One-Out, per quanto teoricamente esatto, nasconde un'insidia algoritmica: esso dipende dai singoli valori assunti dagli elementi sulla diagonale principale della matrice di proiezione P . Se per alcune osservazioni P_{ii} assume un valore prossimo allo zero, il termine corrispondente nella matrice $(\text{diag}(P))^{-2}$ diverge. Questo porta la stima della varianza $\hat{\sigma}_{LOO}^2$ a esplodere numericamente, rischiando di far scartare un'ottima architettura solo a causa di una singola configurazione sfavorevole tra un dato e un centro.

Per ovviare a questa criticità matematica, si ricorre al criterio della **Generalized Cross-Validation** (GCV). L'intuizione fondamentale alla base di questo metodo è un'operazione di smoothing: si eliminano le fluttuazioni individuali del denominatore sostituendo ogni singolo elemento della diagonale con il suo valore medio globale.

Ricordando che la somma degli elementi diagonali di una matrice quadrata coincide con la sua traccia ($\text{tr}(P)$), il valore medio della diagonale su un dataset di p campioni risulta essere $\frac{\text{tr}(P)}{p}$. Da un punto di vista matriciale, questo equivale a sostituire la matrice esatta $\text{diag}(P)$ presente nell'equazione della LOO-CV con una matrice diagonale, proporzionale alla matrice identità:

$$\text{diag}(P) \approx \frac{\text{tr}(P)}{p} I_p. \quad (5.8)$$

Sostituendo questa approssimazione nella formula ricavata precedentemente per lo stimatore della varianza, possiamo derivare l'espressione analitica della GCV in modo esplicito:

$$\hat{\sigma}_{GCV}^2 = \frac{1}{p} \hat{\mathbf{y}}^T P \left(\frac{\text{tr}(P)}{p} I_p \right)^{-2} P \hat{\mathbf{y}}. \quad (5.9)$$

Per risolvere questa equazione, calcoliamo innanzitutto l'inverso del quadrato dello scalare tra parentesi:

$$\left(\frac{\text{tr}(P)}{p} \right)^{-2} = \frac{p^2}{(\text{tr}(P))^2}. \quad (5.10)$$

Poiché questo termine è uno scalare, possiamo portarlo fuori dai prodotti matriciali. Inoltre, ricordando che la matrice identità non altera il prodotto ($P I_p P = P^2$), l'equazione si semplifica:

$$\hat{\sigma}_{GCV}^2 = \frac{1}{p} \frac{p^2}{(\text{tr}(P))^2} \hat{\mathbf{y}}^T P I_p P \hat{\mathbf{y}} = \frac{p \hat{\mathbf{y}}^T P^2 \hat{\mathbf{y}}}{(\text{tr}(P))^2}. \quad (5.11)$$

Questa rappresenta l'esatta espressione matriciale della *Generalized Cross-Validation*. La sua vera potenza si rivela quando la traduciamo in termini delle grandezze di sistema discusse nei capitoli precedenti. Sappiamo infatti che il numeratore $\hat{\mathbf{y}}^T P^2 \hat{\mathbf{y}}$ rappresenta l'ordinaria Somma degli Errori al Quadrato (S) calcolata sui dati di addestramento.

Allo stesso tempo, la traccia della matrice di proiezione è direttamente legata al numero effettivo di parametri (i gradi di libertà impiegati dalla rete, γ) tramite la relazione fondamentale $\text{tr}(P) = p - \gamma$. Sostituendo questi due termini, l'equazione assume la sua forma finale:

$$\hat{\sigma}_{GCV}^2 = \frac{pS}{(p - \gamma)^2}. \quad (5.12)$$

Da questa formulazione emerge con la dinamica del compromesso bias-varianza. Il numeratore è guidato dall'errore di addestramento S , che tende a diminuire inesorabilmente mano a mano che aggiungiamo funzioni di base al modello. Il denominatore, agisce come una penalità per la complessità strutturale: man mano che la rete aumenta i gradi di libertà effettivi (γ), il termine $(p - \gamma)^2$ decresce rapidamente, spingendo verso l'alto il valore di $\hat{\sigma}_{GCV}^2$.

Di conseguenza, gli algoritmi di costruzione automatica della rete utilizzeranno questo punteggio come indicatore fondamentale: aggiungeranno nuovi centri fintanto che la riduzione dell'errore giustificherà il costo in termini di complessità (facendo diminuire lo stimatore $\hat{\sigma}_{GCV}^2$), per poi interrompere istantaneamente la crescita del modello non appena la GCV toccherà il suo valore minimo.

Algoritmi di Ottimizzazione e Selezione Strutturale

Nel capitolo precedente abbiamo definito la Generalized Cross-Validation (GCV) come criterio fondamentale per stimare le prestazioni della rete su campioni non osservati. Avendo a disposizione questo potente strumento di valutazione, il focus si sposta ora sulle procedure algoritmiche necessarie per costruire materialmente il modello ottimale. L'ottimizzazione di una rete a base radiale si articola su due fronti: la calibrazione della regolarizzazione (per il controllo del valore dei pesi) e la selezione strutturale (per determinare il numero esatto e la posizione dei centri).

6.1 Ottimizzazione della Regolarizzazione (Ridge Regression)

6.1.1 La Ridge Regression

L'addestramento standard di una rete a base radiale mira a minimizzare esclusivamente la Somma degli Errori al Quadrato (S) sui dati di addestramento. Come discusso nel primo capitolo, tuttavia, la ricerca ossessiva di minimizzazione di S espone il modello al grave rischio di *overfitting*. Dal punto di vista algebrico, questo accade quando le funzioni di base si sovrappongono eccessivamente o il loro numero si avvicina al numero delle osservazioni disponibili (p), la matrice del sistema $H^T H$ diventa mal condizionata. L'inversione diretta di tale matrice assegna ai pesi $\mathbf{w}$ valori spropositati e di segno opposto, generando una funzione di interpolazione altamente oscillante, instabile e del tutto priva di utilità predittiva.

Per stabilizzare numericamente il sistema e limitare la libertà di fluttuazione del modello, si ricorre alla tecnica della regolarizzazione. Nel Capitolo 3 abbiamo introdotto una matrice di penalità locale Λ . In questo capitolo ci concentreremo sulla sua forma computazionalmente efficiente, nota in statistica come **Ridge Regression** (o regolarizzazione di Tikhonov). L'intuizione fondamentale consiste nel semplificare il problema adottando un unico parametro di penalità globale λ , che viene moltiplicato per il quadrato della norma vettoriale dei pesi.

La nuova funzione di costo regolarizzata, che indicheremo con C , assume la seguente forma scalare:

$$C = \sum_{i=1}^p (\hat{y}_i - f_i(\mathbf{x}_i))^2 + \lambda \sum_{j=1}^m w_j^2. \quad (6.1)$$

In forma matriciale, definendo il vettore dei residui come $\mathbf{e} = \hat{\mathbf{y}} - H\mathbf{w}$, l'equazione può essere riscritta in modo compatto:

$$C = \mathbf{e}^T \mathbf{e} + \lambda \mathbf{w}^T \mathbf{w}. \quad (6.2)$$

Il termine $\lambda > 0$ prende il nome di **parametro di regolarizzazione globale**. Esso agisce come un parametro di controllo che governa il compromesso bias-varianza del modello:

- Se $\lambda \rightarrow 0$, la forza della penalità si annulla e si ricade nel caso limite della regressione standard (Minimi Quadrati Ordinari), con il conseguente elevato rischio di overfitting.
- Se $\lambda \rightarrow \infty$, la penalità domina incontrastata la funzione di costo, forzando tutti i pesi ad assumere un valore tendente a zero, appiattendo la rete in una condizione di grave underfitting.

L'obiettivo dell'addestramento diventa quindi la minimizzazione di questa specifica funzione di costo C rispetto ai pesi $\mathbf{w}$. Calcolando il gradiente vettoriale di C e imponendo la condizione di stazionarietà (esattamente come derivato nel Capitolo 3, ponendo $\Lambda = \lambda I_m$), si ottiene la formulazione finale dell'Equazione Normale regolarizzata:

$$(H^T H + \lambda I_m) \mathbf{w} = H^T \hat{\mathbf{y}}. \quad (6.3)$$

In questa equazione, I_m rappresenta la matrice identità di dimensione $m \times m$.

Inoltre poiché la matrice originaria $H^T H$ è di per sé semi-definita positiva (potenzialmente singolare), l'aggiunta del termine λI_m trasla l'intero spettro dei suoi autovalori di una quantità pari a λ . Pertanto, per qualsiasi valore di penalità strettamente positivo ($\lambda > 0$), questa operazione garantisce che la nuova matrice del sistema sia definita positiva e, di conseguenza, sempre invertibile.

Denotando la nuova matrice del sistema stabilizzato con $\mathbf{A} = H^T H + \lambda I_m$, la soluzione per il vettore dei pesi ottimali risulta essere:

$$\hat{\mathbf{w}} = \mathbf{A}^{-1} H^T \hat{\mathbf{y}}. \quad (6.4)$$

Rispetto alla derivazione affrontata nel Capitolo 3 (in cui la regolarizzazione era affidata a una matrice Λ contenente potenzialmente un parametro λ_j diverso per ogni funzione di base), la *Ridge Regression* introduce un'importante semplificazione. Adottando un singolo parametro globale λ uguale per tutti i centri, trasformiamo il problema di ottimizzazione: non siamo più costretti a ottimizzare contemporaneamente m parametri distinti, ma possiamo limitarci a esplorare uno spazio monodimensionale. Sfruttando questa efficienza, potremo generare in frazioni di secondo diverse configurazioni della rete al variare dello scalare λ , lasciando che sia la curva della GCV a decretare quale sia il livello di regolarizzazione perfetto.

6.1.2 Ottimizzazione del Parametro λ

L'introduzione della *Ridge Regression* risolve il problema dell'instabilità numerica, ma trasferisce la difficoltà decisionale su un altro livello: come scegliere il valore ottimale del parametro λ in modo oggettivo?

La determinazione del parametro λ rappresenta a tutti gli effetti un problema di ottimizzazione. Avendo formalizzato la Generalized Cross-Validation ($\hat{\sigma}_{GCV}^2$) come strumento affidabile per misurare la capacità di predizione della rete, l'obiettivo matematico diventa la minimizzazione di tale criterio rispetto allo scalare λ :

$$\lambda_{opt} = \arg \min_{\lambda} \left(\frac{pS(\lambda)}{(p - \gamma(\lambda))^2} \right). \quad (6.5)$$

Si noti come sia l'errore sui dati di addestramento S , sia i gradi di libertà effettivi γ , siano funzioni dirette del parametro di regolarizzazione λ . Per individuare il minimo analitico di questa funzione, è necessario calcolare la derivata prima della GCV rispetto a λ e imporla uguale a zero:

$$\frac{\partial}{\partial \lambda} \hat{\sigma}_{GCV}^2 = 0. \quad (6.6)$$

La derivazione analitica richiede l'applicazione della regola della catena sulle matrici inverse, poiché λ è annidato all'interno della matrice del sistema $\mathbf{A}$ e, di conseguenza, all'interno della matrice di proiezione P . Sviluppando i passaggi algebrici, l'azzeramento della derivata non restituisce una soluzione in forma chiusa, ma genera un'equazione implicita (di punto fisso). Il parametro ottimo deve infatti soddisfare la seguente uguaglianza:

$$\lambda = \frac{\gamma \hat{\sigma}_{GCV}^2}{\hat{\mathbf{w}}^T \hat{\mathbf{w}}}. \quad (6.7)$$

Questa equazione rivela una dinamica fondamentale del modello: il fattore di regolarizzazione ottimale è direttamente proporzionale alla varianza stimata del rumore ($\hat{\sigma}_{GCV}^2$) e ai gradi di libertà impiegati (γ), ed è inversamente proporzionale alla norma quadrata dei pesi della rete ($\hat{\mathbf{w}}^T \hat{\mathbf{w}}$).

Poiché le variabili al membro di destra dipendono a loro volta dallo stesso λ , la risoluzione richiede un approccio numerico. L'equazione implicita si trasforma in una formula iterativa:

$$\lambda^{(k+1)} = \frac{\gamma^{(k)} (\hat{\sigma}_{GCV}^2)^{(k)}}{(\hat{\mathbf{w}}^{(k)})^T \hat{\mathbf{w}}^{(k)}}. \quad (6.8)$$

Operativamente, l'algoritmo di ottimizzazione procede secondo i seguenti passi:

1. **Inizializzazione:** Si assegna a $\lambda^{(0)}$ un valore arbitrario di partenza (tipicamente un valore molto piccolo).
2. **Calcolo del Modello:** Fissato $\lambda^{(k)}$, si costruisce la matrice del sistema $\mathbf{A}$, si calcolano i pesi ottimali $\hat{\mathbf{w}}^{(k)}$ e si valutano $\gamma^{(k)}$ e $(\hat{\sigma}_{GCV}^2)^{(k)}$.
3. **Aggiornamento:** Si utilizza la formula iterativa per calcolare il nuovo parametro $\lambda^{(k+1)}$.
4. **Condizione di Arresto:** Si ripete il ciclo dal passo 2 fino a quando la variazione $|\lambda^{(k+1)} - \lambda^{(k)}|$ scende al di sotto di una soglia di tolleranza prefissata, decretando la convergenza numerica.

Questo ciclo iterativo converge rapidamente (in genere in pochissime iterazioni) e garantisce l'individuazione del parametro di regolarizzazione che minimizza la GCV.

6.1.3 Local Ridge Regression

Nel Capitolo 3 avevamo introdotto, la possibilità di utilizzare una matrice di penalità diagonale Λ , dotata di un parametro indipendente λ_j per ciascuna funzione di

base. In quel caso ci eravamo limitati a definirne l'effetto algebrico sull'Equazione Normale. Ora che abbiamo derivato la metrica GCV e compreso l'ottimizzazione di un parametro globale (nella *Ridge Regression* standard), possiamo riprendere quell'intuizione teorica e trasformarla in uno strumento pratico.

L'approccio visto nel paragrafo precedente (un singolo parametro λ) è ragionevole quando non abbiamo motivo di preferire alcune funzioni di base ad altre. Tuttavia, nella pratica, alcune basi modellano la reale struttura della funzione sottostante, mentre altre insistono su aree dominate dal rumore o risultano quasi irrilevanti.

Penalizzarle tutte con la stessa intensità è quindi una scelta conservativa che rischia di limitare la flessibilità del modello. La generalizzazione naturale di questo concetto consiste nell'assegnare un parametro di regolarizzazione λ_j distinto a ciascuna funzione di base, ottenendo la seguente funzione di costo:

$$C(\mathbf{w}) = \sum_{i=1}^p (\hat{y}_i - f_i(\mathbf{x}_i))^2 + \sum_{j=1}^m \lambda_j w_j^2. \quad (6.9)$$

Questa espressione coincide con quella della *Ridge Regression* standard quando tutti i parametri di penalità sono identici, ovvero $\lambda_j = \lambda$ per ogni j . Introducendo la matrice diagonale delle penalità $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_m)$, la matrice del sistema diventa:

$$\mathbf{A} = H^T H + \Lambda, \quad (6.10)$$

e le espressioni analitiche del vettore dei pesi ottimali e della matrice di proiezione dei residui rimangono formalmente identiche al caso standard:

$$\hat{\mathbf{w}} = \mathbf{A}^{-1} H^T \hat{\mathbf{y}}, \quad (6.11)$$

$$P = I_p - H \mathbf{A}^{-1} H^T, \quad (6.12)$$

con l'unica differenza strutturale che il termine scalare λI_m è stato sostituito dalla matrice Λ . I gradi di libertà effettivi rimangono definiti come:

$$\gamma = \text{tr}(P), \quad (6.13)$$

e il criterio GCV si applica in modo del tutto analogo, con la differenza che ora l'ottimizzazione del vettore dei parametri di regolarizzazione richiede una procedura iterativa.

Inoltre il termine *locale* si giustifica osservando che, se si utilizzano funzioni a base radiale (come le gaussiane), ciascuna di esse è "attiva" solo in una regione limitata dello spazio di input. Di conseguenza, i parametri λ_j controllano la regolarità del modello in modo adattivo: regioni con andamenti complessi beneficiano di λ_j piccoli, che lasciano ai pesi maggiore libertà di adattamento ai dati; al contrario, regioni piatte o dominate dal rumore assumono λ_j grandi, che stabilizzano il modello schiacciando i pesi e riducendo il rischio di *overfitting*. La *Ridge Regression* standard, disponendo di un solo paramtro λ globale, fatica a gestire fenomeni con regolarità mista, costringendo la rete a un sub-ottimale compromesso.

Ottimizzazione Iterativa dei Parametri di Regolarizzazione

Disporre di un parametro λ_j per ciascuna funzione impone un costo algoritmico costoso: invece di un singolo scalare, ci troviamo a dover ottimizzare un vettore di m parametri interdipendenti. Bisogna quindi avvalersi della GCV introdotta in precedenza per determinare i valori ottimali $\hat{\lambda}_j$ in modo efficiente.

Il punto di partenza consiste nel rendere esplicita la dipendenza della matrice di proiezione P dal singolo parametro isolato λ_j . Questa relazione emerge applicando nuovamente le operazioni incrementali di Sherman-Morrison studiate nel Capitolo 4. La proiezione del modello completo è collegata alla proiezione del modello privato della j -esima funzione di base dalla seguente identità:

$$P = P_{(-j)} - \frac{P_{(-j)} \mathbf{h}_j \mathbf{h}_j^T P_{(-j)}}{\tilde{\lambda}_j}, \quad (6.14)$$

dove $P_{(-j)}$ è la matrice di proiezione calcolata escludendo il centro j -esimo, $\mathbf{h}_j$ è la corrispondente j -esima colonna della matrice di design H , e si è introdotta la quantità scalare ausiliaria:

$$\tilde{\lambda}_j = \lambda_j + \mathbf{h}_j^T P_{(-j)} \mathbf{h}_j. \quad (6.15)$$

Soluzione Analitica per $\hat{\lambda}_j$ Ottimale. Sostituendo questa scomposizione di P all'interno della formula originaria della GCV, e mantenendo fissi tutti gli altri parametri λ_k (con $k \neq j$), possiamo raggruppare i termini introducendo cinque quantità scalari. Queste costanti (a, b, c, α, β) dipendono unicamente da $P_{(-j)}$, da $\mathbf{h}_j$ e dai dati osservati $\hat{\mathbf{y}}$, risultando del tutto indipendenti dal parametro λ_j :

$$a = \hat{\mathbf{y}}^T P_{(-j)}^2 \hat{\mathbf{y}}, \quad (6.16)$$

$$b = (\hat{\mathbf{y}}^T P_{(-j)}^2 \mathbf{h}_j)(\hat{\mathbf{y}}^T P_{(-j)} \mathbf{h}_j), \quad (6.17)$$

$$c = (\mathbf{h}_j^T P_{(-j)}^2 \mathbf{h}_j)(\hat{\mathbf{y}}^T P_{(-j)} \mathbf{h}_j)^2, \quad (6.18)$$

$$\alpha = \text{tr}(P_{(-j)}), \quad (6.19)$$

$$\beta = \mathbf{h}_j^T P_{(-j)}^2 \mathbf{h}_j. \quad (6.20)$$

Sfruttando queste sostituzioni, l'equazione della GCV diventa un polinomio razionale di grado 2 nella variabile $\tilde{\lambda}_j$:

$$\text{GCV}(\lambda_j) = \frac{1}{p} \cdot \frac{a\tilde{\lambda}_j^2 - b\tilde{\lambda}_j + c}{(\alpha\tilde{\lambda}_j - \beta)^2}. \quad (6.21)$$

Calcolando la derivata di questa funzione rispetto a λ_j e imponendo la condizione di stazionarietà, la radice ottimale si suddivide in quattro casistiche distinte:

$$\hat{\lambda}_j = \begin{cases} +\infty & \text{se } a\beta = \alpha b, \\ +\infty & \text{se } \mathbf{h}_j^T P_{(-j)} \mathbf{h}_j > \frac{c\alpha - b\beta}{b\alpha - a\beta} \text{ e } a\beta > \alpha b, \\ 0 & \text{se } \mathbf{h}_j^T P_{(-j)} \mathbf{h}_j > \frac{c\alpha - b\beta}{b\alpha - a\beta} \text{ e } a\beta < \alpha b, \\ \frac{c\alpha - b\beta}{b\alpha - a\beta} - \mathbf{h}_j^T P_{(-j)} \mathbf{h}_j & \text{se } \mathbf{h}_j^T P_{(-j)} \mathbf{h}_j \leq \frac{c\alpha - b\beta}{b\alpha - a\beta}. \end{cases} \quad (6.22)$$

Pruning Automatico delle Funzioni di Base. Nel caso in cui la formula restituisce $\hat{\lambda}_j = +\infty$, il peso associato viene azzerato e la formula di aggiornamento della matrice di proiezione mostra che $P \rightarrow P_{(-j)}$. In altre parole, la rete si comporta esattamente come se quel centro non esistesse. Grazie a questo meccanismo, la *Local Ridge Regression* esegue implicitamente un'operazione di *pruning*: le funzioni di base giudicate matematicamente irrilevanti dalla GCV vengono soppresse in modo automatico durante l'ottimizzazione, senza la necessità di ricorrere a euristiche di selezione esterne.

Strategia Iterativa e Inizializzazione. Poiché i parametri λ_j sono accoppiati, l'aggiornamento di un singolo λ_j sposta il punto di minimo di tutti gli altri. La strategia risolutiva consiste nell'ottimizzare un parametro alla volta, aggiornando istantaneamente la matrice P dopo ogni modifica, per poi ripetere il ciclo sull'intero set di basi finché il punteggio globale GCV smette di decrescere. Poiché ogni singolo aggiornamento è progettato per minimizzare (o al più lasciare invariata) la GCV, l'algoritmo gode di convergenza monotona, seppur con la possibilità di potersi arrestare in un minimo locale.

Per mitigare il rischio di convergenza verso soluzioni sub-ottimali o degeneri, è fondamentale non affidarsi ad un'inizializzazione casuale. Due strategie suggeriscono di:

- Calcolare preventivamente il parametro ottimale λ^* della *Ridge Regression* standard globale e utilizzarlo come valore iniziale, ponendo $\lambda_j = \lambda^*$ per ogni $j = 1, \dots, m$.
- Sfruttare i risultati di un algoritmo di costruzione progressiva (come la *Forward Selection*, trattata nella sezione seguente) per isolare un sottoinsieme di centri ad alta confidenza, assegnando un valore iniziale di $\lambda_j = 0$ a tali centri e $\lambda_j = +\infty$ (ovvero escludendole temporaneamente) a tutte le restanti basi candidate.

6.2 Forward Selection

La *Ridge Regression*, nelle sue varianti scalare e locale, opera su un modello di dimensione fissa: il numero di funzioni di base m è stabilito a priori e la regolarizzazione si occupa di controllarne la complessità effettiva. Esiste tuttavia una strategia alternativa, definita *subset selection*, che affronta il problema da una prospettiva differente: invece di utilizzare tutte le M funzioni di base candidate applicando una penalità ai pesi, si ricerca attivamente il sottoinsieme ottimale di basi in grado di produrre il miglior potere predittivo.

Individuare il sottoinsieme ideale tra tutte le combinazioni possibili è un problema combinatorio di tipo NP-hard: con M funzioni candidate, esistono 2^M sottoinsiemi potenziali, il che rende la ricerca computazionalmente costosa anche per valori moderati di M . Si rende quindi necessario l'impiego di euristiche. La più diffusa è la **Forward Selection**: si parte da un modello privo di centri e si aggiunge una funzione di base alla volta, scegliendo ad ogni passo quella che garantisce la maggiore

riduzione dell'errore quadratico, finché un criterio di *model selection* come la GCV smette di decrescere. Questa procedura è generalmente preferibile alla sua speculare Backward Elimination (che parte dal modello completo e rimuove iterativamente le basi meno significative) poiché evita di dover gestire matrici di dimensioni iniziali elevate, che risultano spesso mal condizionate o costose da invertire nel primo passo dell'algoritmo."

6.2.1 Algoritmo e Criterio di Selezione

Ad ogni passo della *Forward Selection*, il modello corrente contiene m funzioni di base, caratterizzate da una matrice di design $H_m \in \mathbb{R}^{p \times m}$ e da una matrice di proiezione dei residui P_m . Si dispone di un insieme complessivo di M centri candidati, le cui risposte sui dati sono rappresentate dai vettori $\{\mathbf{h}_j\}_{j=1}^M$, di cui $M - m$ risultano ancora disponibili per la selezione.

L'obiettivo è individuare la funzione di base j che, se integrata nel modello, massimizza il miglioramento delle prestazioni. Quando si aggiunge il j -esimo candidato, la matrice di proiezione viene aggiornata secondo la formula derivata nel Capitolo 4:

$$P_{m+1} = P_m - \frac{P_m \mathbf{h}_j \mathbf{h}_j^T P_m}{\Delta_j}, \quad (6.23)$$

dove $\Delta_j = \mathbf{h}_j^T P_m \mathbf{h}_j$ (assumendo in questa fase una regolarizzazione nulla per la selezione dei centri). Questa struttura permette di aggiornare lo stato della rete senza dover ricalcolare l'inversa della matrice del sistema da zero.

Dalla formula di aggiornamento si ricava direttamente l'entità della riduzione della Somma degli Errori al Quadrato (S) associata all'aggiunta del candidato j :

$$S_m - S_{m+1} = \frac{(\hat{\mathbf{y}}^T P_m \mathbf{h}_j)^2}{\mathbf{h}_j^T P_m \mathbf{h}_j}. \quad (6.24)$$

Questa quantità viene calcolata per tutti i candidati j non ancora selezionati. L'algoritmo sceglie il centro che massimizza tale differenza, lo integra nella matrice di design H_{m+1} e procede al passo successivo.

Criterio di Arresto. Poiché l'errore di addestramento S è una funzione monotona non crescente rispetto al numero di parametri, esso non può essere utilizzato come segnale di stop (porterebbe immancabilmente a selezionare tutti gli M centri, causando *overfitting*). Si monitora invece l'andamento della GCV: finché il valore della GCV decresce, l'aggiunta di complessità è giustificata da un reale incremento della capacità di predizione. Quando la GCV inizia ad aumentare, significa che il modello sta iniziando ad interpolare il rumore; la procedura viene quindi interrotta e viene restituita l'architettura corrispondente al minimo globale della GCV osservato durante la progressione.

6.2.2 Orthogonal Least Squares (OLS)

La *Forward Selection* descritta in precedenza richiede, ad ogni passo, il calcolo dei termini $\mathbf{h}_j^T P_m \mathbf{h}_j$ e $\hat{\mathbf{y}}^T P_m \mathbf{h}_j$ per tutti gli $M - m$ candidati rimanenti. Poiché ciascu-

no di questi prodotti ha un costo computazionale dell'ordine di $\mathcal{O}(p^2)$ operazioni, il processo può risultare estremamente costoso per dataset di grandi dimensioni. La tecnica degli **Orthogonal Least Squares** (OLS) permette di abbattere drasticamente questo costo a $\mathcal{O}(p)$ operazioni per candidato, garantendo esattamente gli stessi risultati in termini di selezione.

L'intuizione alla base dell'OLS consiste nel mantenere le colonne della matrice di design del sottoinsieme corrente a due a due ortogonali. A tale scopo, la matrice di disegno viene fattorizzata come:

$$H_m = \tilde{H}_m U_m, \quad (6.25)$$

dove $\tilde{H}_m = [\tilde{\mathbf{h}}_1, \dots, \tilde{\mathbf{h}}_m] \in \mathbb{R}^{p \times m}$ possiede colonne a due a due ortogonali ($\tilde{\mathbf{h}}_i^T \tilde{\mathbf{h}}_j = 0$ per $i \neq j$) e $U_m \in \mathbb{R}^{m \times m}$ è una matrice triangolare superiore con elementi unitari sulla diagonale. Questa scomposizione corrisponde, di fatto, all'applicazione del procedimento di ortogonalizzazione di Gram-Schmidt (o fattorizzazione QR) alla matrice di disegno corrente.

Quando si valuta l'aggiunta della j -esima funzione di base candidata, la componente di $\mathbf{h}_j$ già contenuta nello spazio lineare delle basi correnti non può contribuire a ridurre ulteriormente l'errore, poiché è una combinazione lineare di funzioni già presenti nel modello. Solo la componente ortogonale a tale spazio risulta rilevante:

$$\tilde{\mathbf{h}}_j = \mathbf{h}_j - \sum_{k=1}^m \frac{\mathbf{h}_j^T \tilde{\mathbf{h}}_k}{\tilde{\mathbf{h}}_k^T \tilde{\mathbf{h}}_k} \tilde{\mathbf{h}}_k. \quad (6.26)$$

Grazie a questa trasformazione, la riduzione della Somma degli Errori al Quadrato (S) si semplifica in una forma più efficiente:

$$S_m - S_{m+1} = \frac{(\hat{\mathbf{y}}^T \tilde{\mathbf{h}}_j)^2}{\tilde{\mathbf{h}}_j^T \tilde{\mathbf{h}}_j}, \quad (6.27)$$

tale operazione richiede solo p prodotti scalari, contro i p^2 richiesti dalla formula non ortogonalizzata.

Efficienza nel monitoraggio della GCV. Un ulteriore vantaggio dell'approccio OLS riguarda il calcolo del criterio di arresto. Grazie all'ortogonalità delle colonne di $\tilde{H}_m$, la traccia della matrice di proiezione dei residui (nel caso non regolarizzato) si calcola in forma chiusa senza dover esplicitare P_m :

$$\text{tr}(P_m) = p - m. \quad (6.28)$$

Di conseguenza, la GCV al passo m diventa:

$$\text{GCV}_m = \frac{p}{(p - m)^2} \left(\hat{\mathbf{y}}^T \hat{\mathbf{y}} - \sum_{j=1}^m \frac{(\hat{\mathbf{y}}^T \tilde{\mathbf{h}}_j)^2}{\tilde{\mathbf{h}}_j^T \tilde{\mathbf{h}}_j} \right). \quad (6.29)$$

Questa formulazione rende il monitoraggio della capacità di predizione computazionalmente trascurabile, poiché i termini della sommatoria sono sottoprodotti già ottenuti durante le fasi di selezione precedenti.

Recupero dei pesi ottimali. Il vettore dei pesi nel sistema ortogonalizzato è definito da:

$$\tilde{\mathbf{w}}_m = (\tilde{H}_m^T \tilde{H}_m)^{-1} \tilde{H}_m^T \hat{\mathbf{y}}, \quad (6.30)$$

dove, data l'ortogonalità, la j -esima componente vale semplicemente $\tilde{w}_j = (\hat{\mathbf{y}}^T \tilde{\mathbf{h}}_j) / (\tilde{\mathbf{h}}_j^T \tilde{\mathbf{h}}_j)$. Per ottenere i pesi $\hat{\mathbf{w}}_m$ riferiti alle funzioni di base originali (non ortogonalizzate), è sufficiente risolvere il sistema triangolare superiore:

$$\hat{\mathbf{w}}_m = U_m^{-1} \tilde{\mathbf{w}}_m. \quad (6.31)$$

Estensioni regolarizzate. La Forward Selection e l'Orthogonal Least Squares possono essere combinate con la Ridge Regression, ottenendo rispettivamente la *Regularised Forward Selection* e la *Regularised Orthogonal Least Squares*. Nel primo caso la formula di aggiornamento della matrice di proiezione viene modificata introducendo il parametro λ , e il parametro di regolarizzazione può essere ri-ottimizzato tramite la formula iterativa dopo ogni aggiunta di una nuova base. Nel secondo caso la regolarizzazione viene adattata alla struttura ortogonalizzata del modello, mantenendo l'efficienza computazionale dell'OLS. Questi approcci combinati permettono di controllare contemporaneamente la struttura del modello (quali basi includere) e il grado di regolarizzazione, ottenendo in pratica risultati migliori rispetto ai metodi non regolarizzati.